

Assignment No. 1

Title

Write a program to implementing and evaluating a Linear Regression

model Aim

To implement and evaluate a Linear Regression model using Python and scikit-learn to predict continuous output values.

Software Requirements

- Python 3.14
- Jupyter Notebook
- Libraries: NumPy, Pandas, Matplotlib, scikit-learn

Hardware Requirements

- Minimum 4 GB RAM
- Intel i3 Processor or above
- 500 MB free disk space

Theory

Linear Regression is a supervised machine learning algorithm used for predicting continuous values. It establishes a linear relationship between independent variable(s) (X) and dependent variable (Y). The model fits a straight line ($Y = mX + c$) that minimizes the error between predicted and actual values using the least squares method.

Procedure

1. Import required libraries.
2. Load the dataset.
3. Split dataset into training and testing sets.
4. Create Linear Regression model.
5. Train the model using training data.
6. Predict output using test data.
7. Evaluate model using performance metrics like MSE and R^2 score.

Controls Used

- `train_test_split()` – Splits data into training and testing sets.
- `LinearRegression()` – Creates Linear Regression model.
- `fit()` – Trains the model.
- `predict()` – Predicts output values.

- `mean_squared_error()` – Calculates error.
- `r2_score()` – Evaluates accuracy.

Program Code

```
# Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
data = pd.read_csv('advertising.csv')
print(data)

# Define the independent variable (feature) and dependent variable (target)
X = data[['TV']]
y = data['Sales']

# Split the data into training and testing sets (80% train, 20% test) X_train, X_test,
y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initialize and train the Linear Regression model
model = LinearRegression()
model.fit(X_train, y_train)

# Make predictions on the test set
y_pred = model.predict(X_test)

# Evaluate the model's performance
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

# Output the evaluation metrics
print(f'Mean Squared Error (MSE): {mse}')
print(f'R-squared (R2) Score: {r2}')

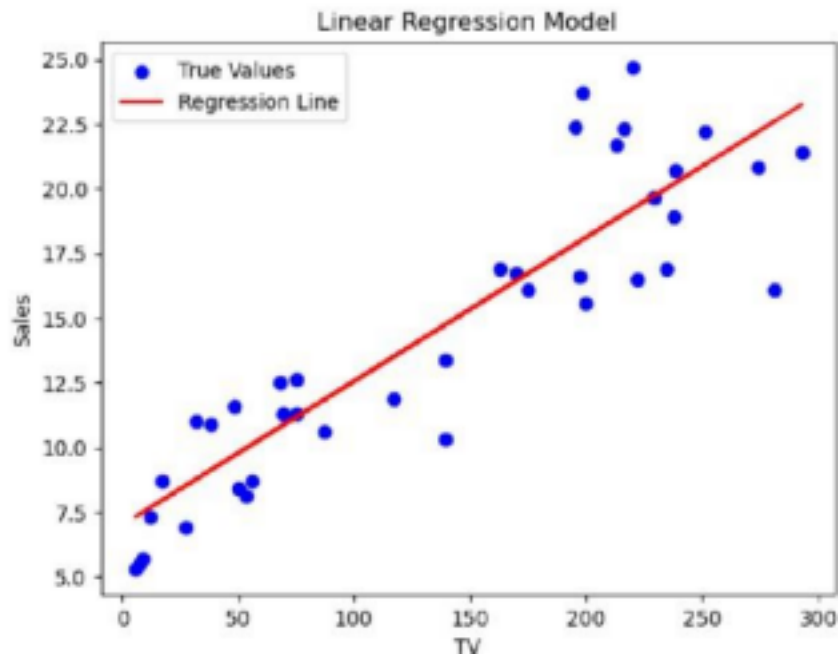
# Visualize the results
plt.scatter(X_test, y_test, color='blue', label='True Values')
plt.plot(X_test, y_pred, color='red', label='Regression Line')
plt.xlabel('TV')
plt.ylabel('Sales')
plt.title('Linear Regression Model')
plt.legend()
```

plt.show()

2

Output

Mean Squared Error (MSE): 6.101072986773964
R-squared (R2) Score: 0.802561303423698



Result

Thus, the Linear Regression model was successfully implemented and evaluated. The model predicts continuous values and its performance was measured using MSE and R^2 score.

Assignment No. 2

Title

Write a program to implementing and evaluating a Logistic Regression

model. Aim

To implement and evaluate a Logistic Regression model using Python and scikit-learn for binary classification problems.

Software Requirements

- Python 3.14
- Jupyter Notebook

- Libraries: NumPy, Pandas, scikit-learn, Matplotlib

Hardware Requirements

3

- Minimum 4 GB RAM
- Intel i3 Processor or above
- 500 MB free disk space

Theory

Logistic Regression is a supervised machine learning algorithm used for classification tasks. Unlike Linear Regression, it predicts categorical outcomes (mainly binary classes such as 0/1, Yes/No). It uses the **Sigmoid (Logistic) function** to convert linear output into probability values between 0 and 1. If the probability is greater than 0.5, the output is classified as 1; otherwise, it is classified as 0. Logistic Regression is widely used in medical diagnosis, spam detection, and credit risk prediction.

Procedure

1. Import required libraries.
2. Create or load the dataset.
3. Separate independent (X) and dependent (Y) variables.
4. Split the dataset into training and testing sets.
5. Create the Logistic Regression model.
6. Train the model using training data.
7. Predict results using test data.
8. Evaluate model using accuracy score and confusion matrix.

Controls Used

- `train_test_split()` – Splits dataset into training and testing data.
- `LogisticRegression()` – Creates logistic regression model.
- `fit()` – Trains the model.
- `predict()` – Predicts output class.
- `accuracy_score()` – Calculates model accuracy.
- `confusion_matrix()` – Displays classification results.
- `classification_report()` – Shows precision, recall, F1-score.

Program Code

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
```

```

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Load dataset from CSV file
df = pd.read_csv('log.csv')

# Assuming the last column is the target variable
X = df.iloc[:, :-1]
y = df.iloc[:, -1]

# Split dataset into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standardize features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train Logistic Regression model
model = LogisticRegression()
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
report = classification_report(y_test, y_pred)

# Print evaluation results
print(f'Accuracy: {accuracy:.4f}')
print('Confusion Matrix:')
print(conf_matrix)
print('Classification Report:')
print(report)

```

Output

```

Accuracy: 0.4650
Confusion Matrix:
[[46 50]
 [57 47]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.45	0.48	0.46	96
1	0.48	0.45	0.47	104
accuracy			0.47	200
macro avg	0.47	0.47	0.46	200
weighted avg	0.47	0.47	0.47	200

5

Result

Thus, the Logistic Regression model was successfully implemented and evaluated. The model classified the dataset into binary categories, and its performance was measured using accuracy, confusion matrix, and classification metrics.

Assignment No. 3

Title

Write a program to implementing and evaluating a Decision Tree classifier.

Aim

To implement and evaluate a Decision Tree Classifier using Python and scikit-learn for classification problems.

Software Requirements

- Python 3.14
- Jupyter Notebook
- Libraries: NumPy, Pandas, scikit-learn, Matplotlib

Hardware Requirements

- Minimum 4 GB RAM
- Intel i3 Processor or above
- 500 MB free disk space

Theory

Decision Tree is a supervised machine learning algorithm used for classification and regression tasks. It works by splitting the dataset into subsets based on the most significant feature at each step. The tree consists of root nodes, internal nodes, branches, and leaf nodes. It uses criteria such as **Entropy (Information Gain)** to determine the best split. Decision Trees are easy to interpret and visualize, making them widely used in medical diagnosis, fraud detection, and decision analysis.

Procedure

1. Import required libraries.
2. Load or create the dataset.
- 6
3. Separate independent (X) and dependent (Y) variables.
4. Split dataset into training and testing sets.
5. Create Decision Tree classifier model.
6. Train the model using training data.
7. Predict results using test data.
8. Evaluate model using accuracy score and confusion matrix.

Controls Used

- `train_test_split()` – Splits dataset into training and testing data.
- `DecisionTreeClassifier()` – Creates Decision Tree model.
- `fit()` – Trains the model.
- `predict()` – Predicts output class.
- `accuracy_score()` – Calculates accuracy.
- `confusion_matrix()` – Displays classification results.
- `classification_report()` – Shows precision, recall, F1-score.

Program Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Load dataset from CSV file
df = pd.read_csv('log.csv')

# Assuming the last column is the target variable
```

```

X = df.iloc[:, :-1]
y = df.iloc[:, -1]

# Split dataset into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standardize features (optional for Decision Tree, but can help with performance)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train Decision Tree model
model = DecisionTreeClassifier()
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)
report = classification_report(y_test, y_pred)

# Print evaluation results
print(f'Accuracy: {accuracy:.4f}')
print('Confusion Matrix:')
print(conf_matrix)
print('Classification Report:')
print(report)

# Visualize the Decision Tree
plt.figure(figsize=(15, 10))
plot_tree(model, filled=True, feature_names=df.columns[:-1],
class_names=str(np.unique(y))) plt.show()

```

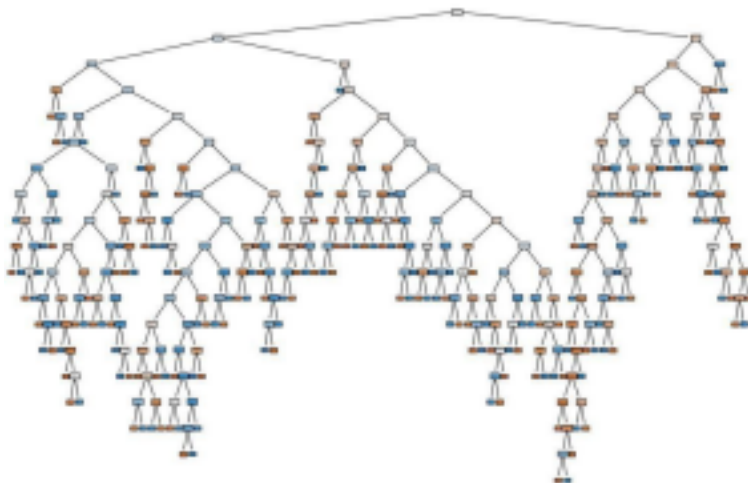
Output


```

Accuracy: 0.4650
Confusion Matrix:
[[45 51]
 [56 48]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.45	0.47	0.46	96
1	0.48	0.46	0.47	104
accuracy			0.47	200
macro avg	0.47	0.47	0.46	200
weighted avg	0.47	0.47	0.47	200



8

Result

Thus, the Decision Tree Classifier was successfully implemented and evaluated. The model classified the dataset based on feature splits and its performance was measured using accuracy, confusion matrix, and classification metrics.

Assignment No. 4

Title

Write a program to implementing Clustering using the K-means

algorithm Aim

To implement clustering using the K-Means algorithm in Python and group similar data points into clusters based on their features.

Software Requirements

- Python 3.14
- Jupyter Notebook
- Libraries: NumPy, Pandas, Matplotlib, scikit-learn

Hardware Requirements

- Minimum 4 GB RAM
- Intel i3 Processor or above
- 500 MB free disk space

Theory

K-Means is an unsupervised machine learning algorithm used for clustering. It divides the dataset into K distinct clusters based on similarity. The algorithm works by initializing K centroids, assigning data points to the nearest centroid using Euclidean distance, and updating centroids by calculating the mean of assigned points. This process repeats until the centroids no longer change significantly. K-Means is widely used in customer segmentation, image compression, and pattern recognition.

Procedure

1. Import required libraries.
2. Create or load the dataset.
3. Select the number of clusters (K).
4. Initialize the K-Means model.
5. Fit the model to the dataset.
6. Predict cluster labels.
7. Visualize the clusters using a scatter plot.

9

Controls Used

- `KMeans()` – Creates K-Means clustering model.
- `fit()` – Trains the clustering model.
- `predict()` – Assigns cluster labels.
- `fit_predict()` – Combines fitting and prediction.
- `cluster_centers_` – Returns centroid coordinates.

Program Code

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
```

```

# Step 1: Generate synthetic data (for demonstration)
# Generating 300 data points with 4 centers (clusters)
X, y = make_blobs(n_samples=300, centers=4, random_state=42)

# Step 2: Apply the K-means algorithm
# Set the number of clusters to 4 (since we generated data with 4 centers)
kmeans = KMeans(n_clusters=4, random_state=42)
kmeans.fit(X)

# Step 3: Get the centroids and labels for the clusters
centroids = kmeans.cluster_centers_
labels = kmeans.labels_

# Step 4: Visualize the clusters
plt.figure(figsize=(8, 6))

# Scatter plot of data points with colors corresponding to cluster labels
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', marker='o',
            edgecolor='k')

# Mark the centroids with a red 'X'
plt.scatter(centroids[:, 0], centroids[:, 1], marker='X', s=200, c='red', label='Centroids')

# Add titles and labels
plt.title('K-means Clustering')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')

# Show the legend
plt.legend()

# Display the plot
plt.show()

```

10

Output



Result

Thus, clustering was successfully implemented using the K-Means algorithm. The dataset was divided into clusters based on similarity, and centroids were identified for each cluster.

Assignment No. 5

Title

Write a program to implementing Dimensionality reduction using

PCA. Aim

To implement dimensionality reduction using Principal Component Analysis (PCA) in Python to reduce the number of features while retaining maximum variance.

Software Requirements

- Python 3.14
- Jupyter Notebook
- Libraries: NumPy, Pandas, Matplotlib, scikit-learn

Hardware Requirements

- Minimum 4 GB RAM
- Intel i3 Processor or above
- 500 MB free disk space

Theory

Principal Component Analysis (PCA) is an unsupervised dimensionality reduction technique used to reduce the number of features in a dataset while preserving maximum variance. It transforms the original correlated variables into a new set of uncorrelated variables called Principal Components. The first principal component captures the highest variance, followed by the second, and so on. PCA is widely used in data visualization, noise reduction, and feature extraction.

Procedure

1. Import required libraries.
2. Load or create the dataset.
3. Standardize the dataset.
4. Create PCA model and select number of components.
5. Fit the PCA model to the dataset.
6. Transform the dataset into principal components.
7. Visualize the reduced data.

Controls Used

- `StandardScaler()` – Standardizes dataset.
- `PCA()` – Creates PCA model.
- `fit()` – Fits the PCA model.
- `transform()` – Transforms data into principal components.
- `fit_transform()` – Combines fitting and transformation.
- `explained_variance_ratio_` – Returns variance explained by each component.

Program Code

```
# Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

12

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
```

```
# Load CSV file (replace 'your_dataset.csv' with your actual dataset file path)
```

```

df = pd.read_csv('log.csv')

# Check the first few rows of the dataset
print(df.head())

# Step 1: Separate features (X) and target (y) if applicable
# Assuming the last column is the target variable
X = df.iloc[:, :-1].values # All rows, all columns except the last one
y = df.iloc[:, -1].values # Last column is the target

# Step 2: Standardize the dataset (important for PCA)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Step 3: Apply PCA to reduce to 2 dimensions for visualization
pca = PCA(n_components=2) # Reduce to 2 components for visualization
X_pca = pca.fit_transform(X_scaled)

# Step 4: Explained variance ratio (how much variance is captured by each component)
print("Explained variance ratio:", pca.explained_variance_ratio_)

# Step 5: Plot the 2D PCA result
plt.figure(figsize=(8,6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='viridis', edgecolor='k', s=100)
plt.title("PCA of Dataset")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.colorbar(label='Target Class')
plt.show()

```

Output





Result

Thus, dimensionality reduction was successfully implemented using PCA. The dataset was reduced to two principal components while retaining maximum variance.

Assignment No. 6

Title

Write a program to implementing Bagging using Random

Forest. Aim

To implement Bagging using the Random Forest algorithm in Python and evaluate its performance for classification.

Software Requirements

- Python 3.14
- Jupyter Notebook
- Libraries: NumPy, Pandas, scikit-learn, Matplotlib

Hardware Requirements

- Minimum 4 GB RAM
- Intel i3 Processor or above
- 500 MB free disk space

Theory

Bagging (Bootstrap Aggregating) is an ensemble learning technique used to improve model performance by combining multiple models. Random Forest is a popular bagging algorithm that creates multiple Decision Trees using random subsets of data and features. Each tree gives a prediction, and the final output is determined by majority voting (for classification) or averaging (for regression). Random Forest reduces overfitting and increases accuracy compared to a single Decision Tree.

Procedure

1. Import required libraries.
2. Load or create the dataset.
3. Separate independent (X) and dependent (Y) variables.
4. Split dataset into training and testing sets.
5. Create Random Forest classifier model.
6. Train the model using training data.
7. Predict results using test data.
8. Evaluate model using accuracy score and confusion matrix.

Controls Used

- `train_test_split()` – Splits dataset into training and testing data.
- `RandomForestClassifier()` – Creates Random Forest model.
- `fit()` – Trains the model.
- `predict()` – Predicts output class.
- `accuracy_score()` – Calculates accuracy.
- `confusion_matrix()` – Displays classification results.
- `classification_report()` – Shows precision, recall, F1-score.

Program Code

```
# Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import LabelEncoder

# Load CSV file (replace 'your_dataset.csv' with your actual dataset file path)
```



```

df = pd.read_csv('log.csv')

# Check the first few rows of the dataset
print(df.head())

# Step 1: Handle missing values (if any)
# Example: Drop rows with missing values (you can also fill with the mean or median)
df = df.dropna()

# Step 2: Separate features (X) and target (y)
# Assuming the last column is the target variable
X = df.iloc[:, :-1].values # All rows, all columns except the last one (features)
y = df.iloc[:, -1].values # Last column is the target

# Step 3: Encode the target variable if it's categorical
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(y)

# Step 4: Split the data into training and testing sets X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.3, random_state=42)

# Step 5: Initialize and train the Random Forest Classifier
rf_classifier = RandomForestClassifier(n_estimators=100, random_state=42)
rf_classifier.fit(X_train, y_train)

# Step 6: Make predictions on the test set
y_pred = rf_classifier.predict(X_test) # Step 7: Evaluate the model's performance
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy of Random Forest on test data: {accuracy *

```

100:.2f}%') **Output**



Result

Thus, Bagging was successfully implemented using the Random Forest algorithm. The ensemble model improved classification performance by combining multiple decision trees, and its performance was evaluated using accuracy and classification metrics.

Assignment No. 7

Title

Write a program to implementing Boosting using AdaBoost

Aim

To implement Boosting using the AdaBoost algorithm in Python and evaluate its performance for classification.

Software Requirements

- Python 3.14
- Jupyter Notebook
- Libraries: NumPy, Pandas, scikit-learn, Matplotlib

Hardware Requirements

- Minimum 4 GB RAM
- Intel i3 Processor or above
- 500 MB free disk space

Theory

Boosting is an ensemble learning technique that combines multiple weak learners to create a strong learner. AdaBoost (Adaptive Boosting) works by training multiple weak classifiers (usually Decision Stumps). After each iteration, it increases the weight of incorrectly classified instances so that the next model focuses more on difficult cases. The final prediction is obtained by combining the weighted predictions of all weak learners. AdaBoost improves accuracy and reduces bias compared to a single model.

Procedure

1. Import required libraries.
2. Load or create the dataset.
3. Separate independent (X) and dependent (Y) variables.

17

4. Split dataset into training and testing sets.
5. Create AdaBoost classifier model.
6. Train the model using training data.
7. Predict results using test data.
8. Evaluate model using accuracy score and confusion matrix.

Controls Used

- `train_test_split()` – Splits dataset into training and testing data.
- `AdaBoostClassifier()` – Creates AdaBoost model.
- `fit()` – Trains the model.
- `predict()` – Predicts output class.
- `accuracy_score()` – Calculates accuracy.
- `confusion_matrix()` – Displays classification results.
- `classification_report()` – Shows precision, recall, F1-score.

Program Code

```
# Import necessary libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import LabelEncoder

# Load CSV file (replace 'your_dataset.csv' with your actual dataset file path)
df = pd.read_csv('log.csv')

# Check the first few rows of the dataset
print(df.head())

# Step 1: Handle missing values (if any)
# Example: Drop rows with missing values (you can also fill with the mean or median)
df = df.dropna()

# Step 2: Separate features (X) and target (y)
# Assuming the last column is the target variable
X = df.iloc[:, :-1].values # All rows, all columns except the last one (features)
y = df.iloc[:, -1].values # Last column is the target
```

Step 3: Encode the target variable if it's categorical

```
label_encoder = LabelEncoder()
```

```
y = label_encoder.fit_transform(y)
```

Step 4: Split the data into training and testing sets

18

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

Step 5: Initialize and train the AdaBoost classifier with a DecisionTree as the base estimator

DecisionTreeClassifier with max_depth=1 is used to create a weak learner (stump)

```
base_estimator = DecisionTreeClassifier(max_depth=1)
```

Update: Use 'estimator' instead of 'base_estimator'

```
adaboost_classifier = AdaBoostClassifier(estimator=base_estimator, n_estimators=50,  
random_state=42)
```

```
adaboost_classifier.fit(X_train, y_train)
```

Step 6: Make predictions on the test set

```
y_pred = adaboost_classifier.predict(X_test)
```

Step 7: Evaluate the model's performance

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print(f'Accuracy of AdaBoost on test data: {accuracy * 100:.2f}%')
```

Output



Result

Thus, Boosting was successfully implemented using the AdaBoost algorithm. The ensemble model improved classification performance by focusing on misclassified instances, and its performance was evaluated using accuracy and classification metrics.

Assignment No. 8

Title

Write a program to implementing SVM for classification tasks.

Aim

To implement a Support Vector Machine (SVM) algorithm in Python for classification tasks and evaluate its performance.

Software Requirements

- Python 3.14
- Jupyter Notebook
- Libraries: NumPy, Pandas, scikit-learn, Matplotlib

Hardware Requirements

- Minimum 4 GB RAM
- Intel i3 Processor or above
- 500 MB free disk space

Theory

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression tasks. In classification, SVM finds the optimal hyperplane that best separates data points of different classes with maximum margin. The data points closest to the hyperplane are called support vectors. SVM can handle both linear and non-linear classification problems using kernel functions such as Linear, Polynomial, and Radial Basis Function (RBF). It is widely used in image recognition, text classification, and bioinformatics.

Procedure

1. Import required libraries.
2. Load or create the dataset.

3. Separate independent (X) and dependent (Y) variables.
4. Split dataset into training and testing sets.
5. Standardize the dataset (important for SVM).
6. Create SVM classifier model.
7. Train the model using training data.
8. Predict results using test data.
9. Evaluate model using accuracy score and confusion matrix.

Controls Used

20

- `train_test_split()` – Splits dataset into training and testing data.
- `StandardScaler()` – Standardizes features.
- `SVC()` – Creates SVM classifier model.
- `fit()` – Trains the model.
- `predict()` – Predicts output class.
- `accuracy_score()` – Calculates accuracy.
- `confusion_matrix()` – Displays classification results.
- `classification_report()` – Shows precision, recall, F1-score.

Program Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Load dataset from CSV file (replace 'your_dataset.csv' with your actual file path)
df = pd.read_csv('pca.csv')

# Check the first few rows of the dataset
print(df.head())

# Assuming the last column is the target variable (classification labels) X =
df.iloc[:, :-1] # Select all rows and all columns except the last one for features y =
df.iloc[:, -1] # Select the last column for the target (labels)

# Step 1: Split the dataset into training and testing sets (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Step 2: Standardize features using StandardScaler (important for SVM)
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train) # Fit on training data and transform it
X_test = scaler.transform(X_test) # Use the same scaler to transform test data
```

```
# Step 3: Train the Support Vector Machine (SVM) model
svm_model = SVC(kernel='linear', random_state=42) # Using a linear kernel for simplicity
svm_model.fit(X_train, y_train) # Train the model on the training set
```

```
# Step 4: Make predictions using the trained SVM model
y_pred = svm_model.predict(X_test)
```

```
# Step 5: Evaluate the model's performance
accuracy = accuracy_score(y_test, y_pred)
```

21

```
conf_matrix = confusion_matrix(y_test, y_pred)
report = classification_report(y_test, y_pred)
```

```
# Print evaluation results
print(f'Accuracy: {accuracy:.4f}')
print('Confusion Matrix:')
print(conf_matrix)
print('Classification Report:')
print(report)
```

```
# Step 6: Optional - Visualize the decision boundaries (only works for 2D features)
# This is just a visualization example for datasets with two features if X.shape[1]
== 2: # Check if we have only two features for visualization
```

```
# Create a mesh grid for plotting decision boundaries
h = .02
x_min, x_max = X_train[:, 0].min() - 1, X_train[:, 0].max() + 1
y_min, y_max = X_train[:, 1].min() - 1, X_train[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
```

```
# Predict over the mesh grid
Z = svm_model.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
```

```
# Plot decision boundary
plt.contourf(xx, yy, Z, alpha=0.8)
plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train, edgecolors='k', marker='o',
            cmap=plt.cm.Paired)
plt.title('SVM Decision Boundary with Linear Kernel')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

Output



 22





Result

Thus, the Support Vector Machine (SVM) classifier was successfully implemented and evaluated. The model classified the dataset using an optimal separating hyperplane, and its performance was measured using accuracy and classification metrics.

23

Assignment No. 9

Title

Write a program to implementing a simple neural network using TensorFlow/Keras.

Aim

To implement a simple Artificial Neural Network (ANN) using TensorFlow/Keras for classification tasks and evaluate its performance.

Software Requirements

- Python 3.14
- Jupyter Notebook
- Libraries: TensorFlow, Keras, NumPy, Pandas, scikit-learn

Hardware Requirements

- Minimum 4 GB RAM
- Intel i3 Processor or above
- 1 GB free disk space

Theory

An Artificial Neural Network (ANN) is a deep learning model inspired by the human brain. It consists of an input layer, one or more hidden layers, and an output layer. Each layer contains neurons connected by weights. The network learns by adjusting these weights using backpropagation and optimization algorithms such as Adam. Activation functions like ReLU and Sigmoid introduce non-linearity. Neural networks are widely used in image recognition, speech processing, and classification problems.

Procedure

1. Import required libraries.
2. Load or create the dataset.
3. Split dataset into training and testing sets.
4. Standardize the dataset.
5. Create a Sequential neural network model.
6. Add input, hidden, and output layers.
7. Compile the model using optimizer and loss function.
8. Train the model using training data.
9. Evaluate the model using test data.

Controls Used

 24

- `Sequential()` – Creates neural network model.
- `Dense()` – Adds fully connected layers.
- `compile()` – Configures optimizer and loss function.
- `fit()` – Trains the model.
- `evaluate()` – Evaluates model performance.
- `predict()` – Predicts output values.

Program Code

```
import pandas as pd
import numpy as np
import tensorflow as tf
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input
import matplotlib.pyplot as plt
df = pd.read_csv('diabetes.csv')
df.head()
print ('Number of Rows :', df.shape[0])
```

```

print ('Number of Columns :', df.shape[1])
print ('Number of Patients with outcome 1 :', df.Outcome.sum())
print ('Event Rate :', round(df.Outcome.mean()*100,2) ,'%')
df.describe()
from sklearn.model_selection import train_test_split
X = df.to_numpy()[0:8]
Y = df.to_numpy()[8]
seed = 42
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size = 0.25, random_state = seed)
print (f'Shape of Train Data : {X_train.shape}')
print (f'Shape of Test Data : {X_test.shape}')
model = Sequential([
    Input(shape=(8,)), # Define the input shape using the new `shape` argument
    Dense(24, activation='relu'),
    Dense(12, activation='relu'),
    Dense(1, activation='sigmoid'),
])
# Compile the model (optional, but necessary for training)
model.compile(optimizer = 'adam', loss = 'binary_crossentropy', metrics = ['accuracy'])
# Summary of the model
#model.summary()
model.summary()
history = model.fit(X_train, y_train, epochs=150, batch_size=32, verbose = 1)
# Plotting loss
plt.plot(history.history['loss'])
plt.title('Binary Cross Entropy Loss on Train dataset')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.show()
# Plotting accuracy metric
plt.plot(history.history['accuracy'])
plt.title('Accuracy on the train dataset')

```

25

```

plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.show()

```

Output





Total params: **529** (2.07 KB)

Trainable params: **529** (2.07 KB)

Non-trainable params: **0** (0.00 B)

Epoch 1/150

18/18 ————— 1s 3ms/step - accuracy: 0.5895 - loss:
4.9180

Epoch 2/150

18/18 ————— 0s 3ms/step - accuracy: 0.5057 - loss:
1.5014

Epoch 3/150

18/18 ————— 0s 3ms/step - accuracy: 0.6015 - loss:
1.1479

Epoch 4/150

18/18 ————— 0s 3ms/step - accuracy: 0.6004 - loss:
1.0280

Epoch 5/150

18/18 ————— 0s 2ms/step - accuracy: 0.6393 - loss:
0.8963

Epoch 6/150

26

18/18 ————— 0s 2ms/step - accuracy: 0.7780 - loss:
0.4835

Epoch 142/150

18/18 ————— 0s 2ms/step - accuracy: 0.7715 - loss:
0.5056

Epoch 143/150

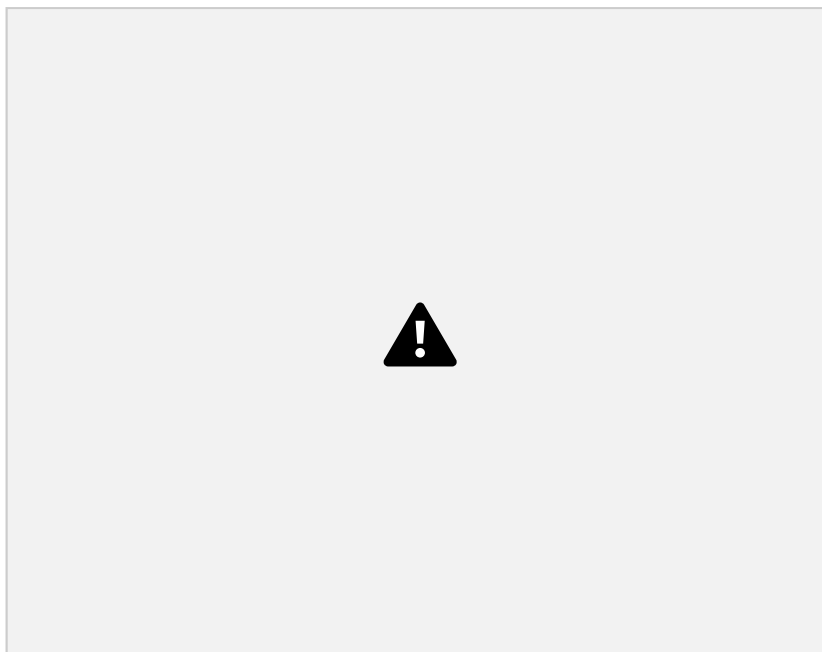
18/18 ————— 0s 2ms/step - accuracy: 0.7542 - loss:
0.5170

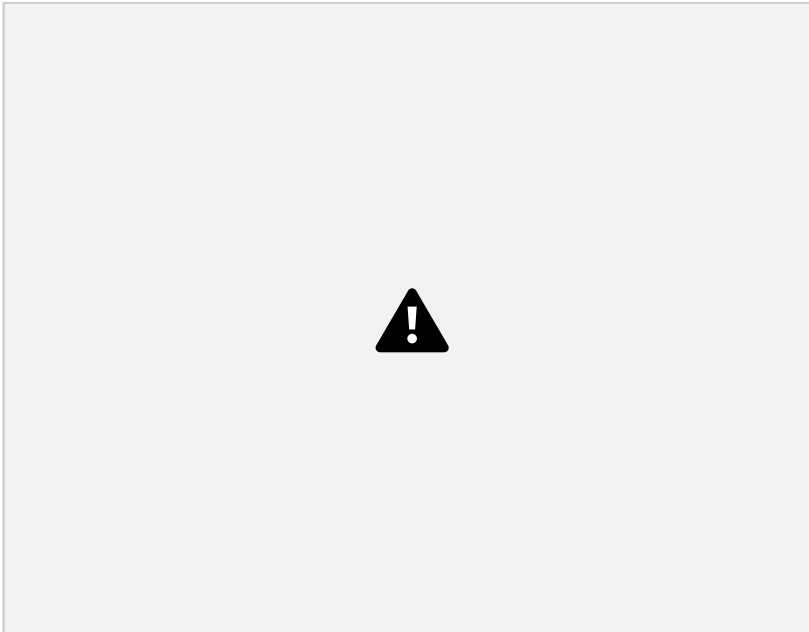
Epoch 144/150

18/18 ————— 0s 2ms/step - accuracy: 0.7890 - loss:
0.4876

Epoch 145/150

18/18 ————— 0s 2ms/step - accuracy: 0.8027 - loss:
0.4567
Epoch 146/150
18/18 ————— 0s 2ms/step - accuracy: 0.7647 - loss:
0.5335
Epoch 147/150
18/18 ————— 0s 2ms/step - accuracy: 0.7290 - loss:
0.5359
Epoch 148/150
18/18 ————— 0s 3ms/step - accuracy: 0.7641 - loss:
0.5116
Epoch 149/150
18/18 ————— 0s 2ms/step - accuracy: 0.7540 - loss:
0.5059
Epoch 150/150
18/18 ————— 0s 2ms/step - accuracy: 0.7521 - loss:
0.5285





Result

Thus, a simple Neural Network was successfully implemented using TensorFlow/Keras. The model was trained and evaluated for binary classification, and its performance was measured using accuracy.

Assignment No. 10

Title

Write a program to implementing with big data concepts using sample datasets & Setting up a Hadoop environment.

Aim

To implement Big Data processing using Hadoop environment setup and perform Word Count using PySpark in Jupyter/Google Colab.

Software Requirements

- Python 3.x
- Jupyter Notebook / Google Colab
- Apache Hadoop (3.3.1)
- PySpark
- Java (OpenJDK 8)

Hardware Requirements

- Minimum 8 GB RAM
- Intel i5 Processor or above
- 10 GB free disk space
- Internet connection (for downloading Hadoop)

Theory

Big Data refers to extremely large datasets that cannot be processed using traditional systems. Hadoop provides distributed storage (HDFS) and distributed processing capabilities. Apache Spark is a fast in-memory distributed computing framework that works with Hadoop. In this experiment, a sample dataset is processed using Spark's RDD operations to perform Word Count. Spark distributes the data across nodes and performs parallel computation using transformations and actions.

Procedure

- Install PySpark.
- Open Jupyter Notebook.
- Create Spark Session.
- Load sample dataset.
- Perform distributed processing (Word Count example).
- Display results.

Controls Used

- `SparkSession.builder()` – Creates Spark session
- `textFile()` – Reads file into RDD
- `flatMap()` – Splits lines into words
- `map()` – Creates key-value pairs
- `reduceByKey()` – Aggregates word counts
- `collect()` – Retrieves results

Program Code

```
# Install Java
!sudo apt update
!sudo apt install openjdk-8-jdk
# Download and extract Hadoop
!wget
http://apache.mirrors.lucidnetworks.net/hadoop/common/hadoop
3.3.1/hadoop-3.3.1.tar.gz
!tar -xvzf hadoop-3.3.1.tar.gz
!mv hadoop-3.3.1 /usr/local/hadoop
dataset = ""
Hadoop is a framework for processing large datasets.
It is used for distributed storage and distributed computing.
```

Hadoop is part of the Big Data ecosystem.

29

Hadoop helps process Big Data.

```
"""
# Save file
with open('dataset.txt', 'w') as f:
    f.write(dataset)

from pyspark.sql import SparkSession

# Start Spark
spark = SparkSession.builder.appName('WordCount').getOrCreate()

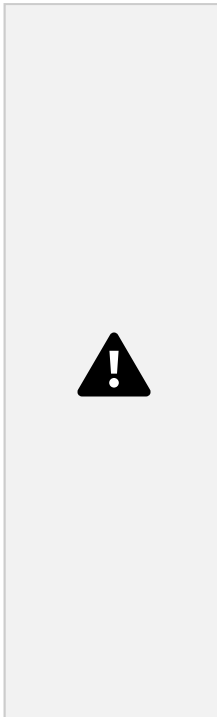
# Read file
rdd = spark.sparkContext.textFile('dataset.txt')

# Word count
word_counts = (
    rdd.flatMap(lambda line: line.split())
        .map(lambda word: (word.lower(), 1))
        .reduceByKey(lambda x, y: x + y)
)

# Print result
for word, count in word_counts.collect():
    print(f"{word}: {count}")

spark.stop()
```

Output



Result

Thus, Big Data processing was successfully implemented using Hadoop setup and PySpark in

Jupyter/Colab. The Word Count program demonstrated distributed storage and parallel processing concepts of Big Data.

30

Assignment No. 11

Title

Write a program to implementing CRUD operations in MongoDB

Aim

To implement CRUD (Create, Read, Update, Delete) operations in MongoDB using Python and PyMongo library.

Software Requirements

- Python 3.x
- MongoDB Server (Running on localhost:27017)
- PyMongo Library
- Jupyter Notebook / VS Code / PyCharm

Hardware Requirements

- Minimum 4 GB RAM
- Intel i3 Processor or above
- 500 MB free disk space

Theory

MongoDB is a NoSQL document-oriented database that stores data in BSON (Binary JSON) format. It is schema-less and flexible. CRUD operations are the basic database operations:

- **Create** → Insert new document
- **Read** → Retrieve document
- **Update** → Modify existing document
- **Delete** → Remove document

PyMongo is a Python library that allows interaction with MongoDB.

Procedure

1. Install PyMongo.
2. Start MongoDB server.
3. Connect Python to MongoDB using `MongoClient`.
4. Create database and collection.

5. Insert a document (Create).
6. Retrieve document (Read).
7. Update document (Update).

31

8. Delete document (Delete).

Controls Used

- `MongoClient()` – Connects to MongoDB server
- `insert_one()` – Inserts single document
- `find_one()` – Retrieves one document
- `update_one()` – Updates document
- `delete_one()` – Deletes document

Program Code

```
!pip install pymongo
from pymongo import MongoClient

# Connect
client = MongoClient("mongodb://localhost:27017/")

# Use existing DB or a new name
db = client['MyDatabase'] # match the existing database case
collection = db['users']

# CREATE
user_data = {
    'name': 'John Doe',
    'age': 30,
    'email': 'john.doe@example.com'
}
result = collection.insert_one(user_data)
print(f"Document inserted with ID: {result.inserted_id}")

# READ
user = collection.find_one({"name": "John Doe"})
print("Found user:", user)

# UPDATE
update_result = collection.update_one(
    {"name": "John Doe"},
    {"$set": {"age": 31}}
)
print(f"Documents matched: {update_result.matched_count}, Documents
modified: {update_result.modified_count}")
```

```
# DELETE
delete_result = collection.delete_one({"name": "John Doe"})
print(f'Documents deleted: {delete_result.deleted_count}')
```

32

Output



```
Document inserted with ID: 69d4a1902d3f78bb01fae491
Found user: {'_id': ObjectId('69d4a1902d3f78bb01fae491'), 'name': 'John
Doe', 'age': 30, 'email': 'john.doe@example.com'}
Documents matched: 1, Documents modified: 1
Documents deleted: 1
```

Result

Thus, CRUD operations were successfully implemented in MongoDB using PyMongo. The program demonstrated insertion, retrieval, updating, and deletion of a document from the MongoDB collection.

Assignment No. 12

Title

Write a program to implementing with NLTK: Tokenization, stemming, and

lemmatization Aim

To implement Natural Language Processing (NLP) tasks such as Tokenization, Stemming, and Lemmatization using the NLTK library in Python.

Software Requirements

- Python 3.x
- Jupyter Notebook / VS Code / PyCharm
- NLTK Library

Hardware Requirements

- Minimum 4 GB RAM
- Intel i3 Processor or above
- 500 MB free disk space

Theory

33

Natural Language Processing (NLP) is a field of Artificial Intelligence that enables computers to understand and process human language. NLTK (Natural Language Toolkit) is a popular Python library used for NLP tasks.

- **Tokenization:** Splitting text into smaller units such as words or sentences.
- **Stemming:** Reducing words to their root form by removing suffixes (may not produce meaningful words).
- **Lemmatization:** Reducing words to their base or dictionary form using vocabulary and morphological analysis.

Procedure

- Install NLTK library.
- Download required NLTK resources.
- Import necessary modules.
- Provide sample text.
- Perform Tokenization.
- Perform Stemming using Porter Stemmer.
- Perform Lemmatization using WordNet Lemmatizer.
- Display results.

Controls Used

- `nltk.download()` – Downloads required resources
- `word_tokenize()` – Splits text into words
- `PorterStemmer()` – Performs stemming
- `WordNetLemmatizer()` – Performs lemmatization

Program Code

```
!pip install nltk
import nltk
# Download the 'punkt' tokenizer
nltk.download('punkt')
import nltk
# Download the 'punkt_tab' resource, which is required for tokenization
nltk.download('punkt_tab')
# Download other necessary resources for lemmatization and stop words
```

```

nltk.download('wordnet')
nltk.download('stopwords')
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer
# Sample text for demonstration
text = "NLTK is a great toolkit for Natural Language Processing. Tokenization, Stemming, and
Lemmatization are important tasks."

```

34

```

# Tokenization: Split text into words
tokens = word_tokenize(text)
print("Tokens:", tokens)
# Stemming: Reduce words to their root form using Porter Stemmer
stemmer = PorterStemmer()
stemmed_words = [stemmer.stem(word) for word in tokens]
print("Stemmed words:", stemmed_words)
# Lemmatization: Reduce words to their base form using WordNet Lemmatizer
lemmatizer = WordNetLemmatizer()
lemmatized_words = [lemmatizer.lemmatize(word) for word in tokens]
print("Lemmatized words (default pos=noun):", lemmatized_words)
# Optional: Lemmatization with POS tagging (verbs, adjectives, etc.)
lemmatized_verbs = [lemmatizer.lemmatize(word, pos='v') for word in tokens]
print("Lemmatized words (as verbs):", lemmatized_verbs)

```

Output



```

[nltk_data] Downloading package punkt to
[nltk_data] C:\Users\IMRD\AppData\Roaming\nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data] C:\Users\IMRD\AppData\Roaming\nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data] C:\Users\IMRD\AppData\Roaming\nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\IMRD\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!

```



Result

Thus, Tokenization, Stemming, and Lemmatization were successfully implemented using the NLTK library. The program demonstrated basic NLP preprocessing techniques.